Templates

Los templates o plantillas en C++ son un formato que permite escribir código genérico (funciones y clases) para que pueda trabajar con diferentes tipos de datos sin necesidad de reescribir la misma lógica. Evitan duplicar funciones o estructuras cuando hacen exactamente lo mismo, pero con tipos distintos (como int, float).

Template de funciones

Podemos separar este tema en dos sub categorías, los template de funciones y los template de clases. Supongamos tenemos la siguiente función:

```
int maximo(int a, int b) {
    return (a > b) ? a : b;
}
```

Y queremos reutilizar para los distintos tipos de datos que tengamos, int, long, doublé, float, etc. Entonces deberíamos escribir una función para cada uno de estos casos. Estariamos aplicando sobrecarga. Pero como el código ejecuta siempre la misma lógica, independientemente del tipo de dato, seria mas adecuado contar con una única función que haga esto.

```
int    maximo(int a, int b)       { return (a > b) ? a : b; }
double maximo(double a, double b) { return (a > b) ? a : b; }
char   maximo(char a, char b)     { return (a > b) ? a : b; }
```

Para eso estaríamos aplicando los templates. Entonces cuando aplicar templates o sobrecarga:

Situación	Qué conviene
Mismo algoritmo, distintos tipos	Template
Algoritmos *distintos* según el tipo	Sobrecarga
Comportamiento especial para un tipo puntual	Sobrecarga (convive con el template)

Podemos notar en esta tabla que si la lógica cumple siempre los mismos pasos, lo ideal seria impletar soluciones con templates en vez de aplicar sobrecarga.

Veamos un ejemplo

```
template <typename T>
T maximo(T a, T b) {
    return (a > b) ? a : b;
}
```

```
int main() {
    cout << maximo(5, 3) << endl;          // T = int
    cout << maximo(2.7, 9.1) << endl;      // T = double
    cout << maximo('a', 'z') << endl;      // T = char
    return 0;
}
```

En el primer bloque de código estamos definiendo un template. ¿que queremos decir? Que la letra T va a corresponder a un tipo de dato a implementar a futuro en la función. La función definida como máximo, compara dos valores genéricos y devuelve el mayor, sea cual sea el tipo de dato recibido en ambas.

template → palabra clave que le avisa al compilador que lo que sigue es una plantilla, no una función común

<**typename T>** → declara un parámetro de tipo llamado T. Es un "tipo que todavía no sé cuál es"

T en la firma → se usa como si fuera un tipo cualquiera (int, double, etc.)

Que pasa si pasamos distintos tipos de datos:

```
maximo(5, 3.7);  // ERROR de compilación
```

El compilador no puede interpretar si los tipos de datos son int o float, no puede compilar. Como solución se debe especificar que tipo de dato es, por ejemplo:

maximo<double>(5, 3.7)

Otra opción sería declarar mas de un tipo de dato genérico:

```
template <typename T1, typename T2>
void mostrarPar(T1 primero, T2 segundo) {
    cout << "(" << primero << ", " << segundo << ")" << endl;
}
```

```
mostrarPar(42, "hola");       // T1 = int, T2 = const char*
mostrarPar(3.14, 'x');        // T1 = double, T2 = char
```